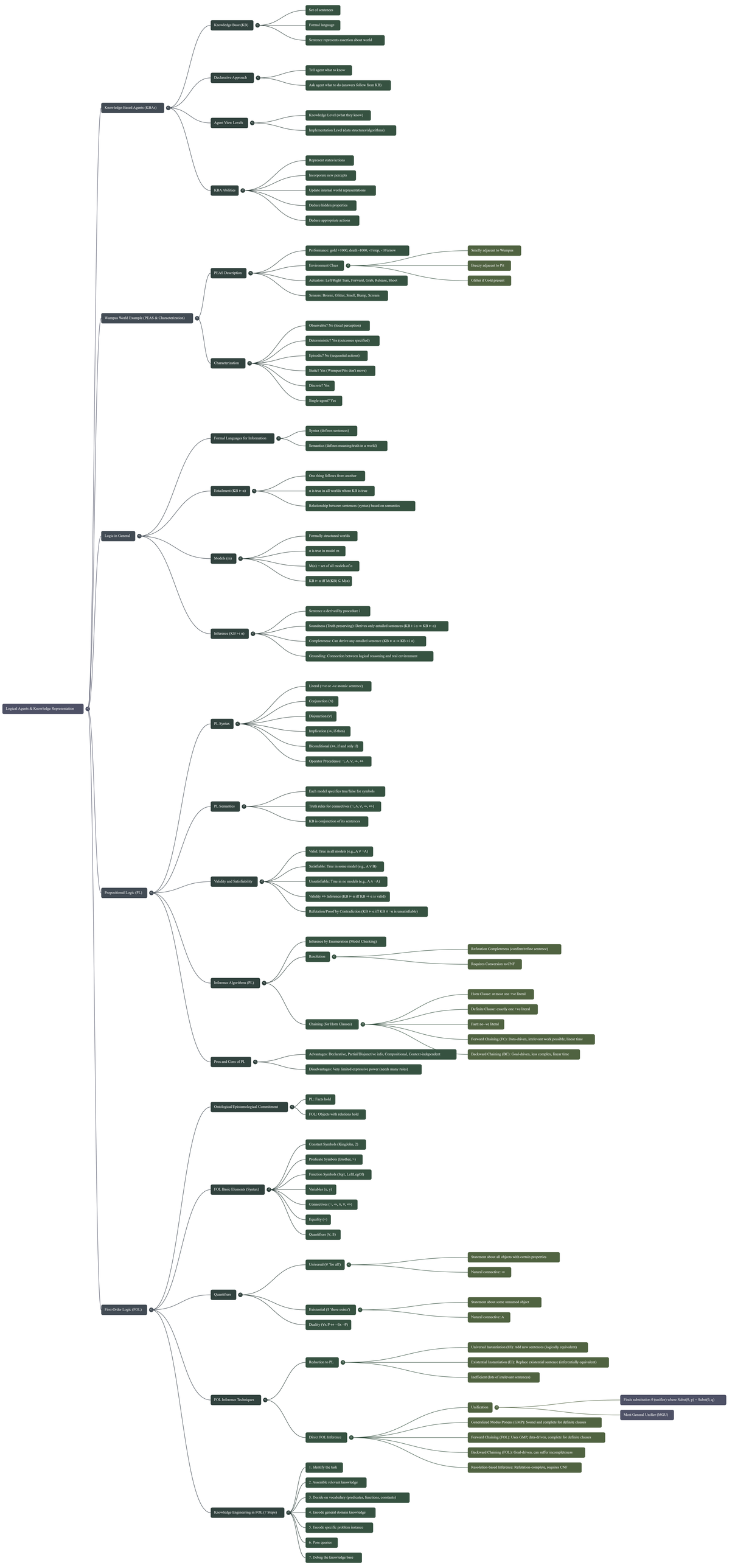
Logical Agents & Knowledge Representation
Knowledge-Based Agents (KBAs)
Knowledge Base (KB)
Set of sentences
Formal language
Sentence represents assertion about world
Declarative Approach
Tell agent what to know
Ask agent what to do (answers follow from KB)
Agent View Levels
Knowledge Level (what they know)
Implementation Level (data structures/algorithms)
KBA Abilities
Represent states/actions
Incorporate new percepts
Update internal world representations
Deduce hidden properties
Dechoc appropriate actions
Wumpus World Example (PEAS & Characterization)
PEAS Description
Performance: gold +1000, death -1000, -1/step, -10/arrow
Environment Clues
Smelly adjacent to Wumpus
Breezy adjacent to Pit
Glitter if Gold present
Actuators: Left/Right Turn, Forward, Grab, Release, Shoot
Sensors: Breeze, Glitter, Smell, Bump, Scream
Characterization
Observable? No (local perception)
Deterministic? Yes (outcomes specified)
Episodic? No (sequential actions)
Static? Yes (Wumpus/Pits don't move)
Discrete? Yes
Single-agent? Yes
Logic in General
Formal Languages for Information
Syntax (defines sentences)
Semantics (defines meaning/truth in a world)
Entailment (KB ⊨ α)
One thing follows from another
α is true in all worlds where KB is true
Relationship between sentences (syntax) based on semantics
Models (m)
Formally structured worlds
α is true in model m
M(α) = set of all models of α
KB ⊨ α iff M(KB) ⊆ M(α)
Inference (KB ⊢i α)
Sentence α derived by procedure i
Soundness (Truth preserving): Derives only entailed sentences (KB ⊢i α ⇒ KB ⊨ α)
Completeness: Can derive any entailed sentence (KB ⊨ α ⇒ KB ⊢i α)
Grounding: Connection between logical reasoning and real environment
Propositional Logic (PL)
PL Syntax
Literal (+ve or -ve atomic sentence)
Conjunction (∧)
Disjunction (∨)
Implication (⇒, if-then)
Biconditional (⇔, if and only if)
Operator Precedence: ¬, ∧, ∨, ⇒, ⇔
PL Semantics
Each model specifies true/false for symbols
Truth rules for connectives (¬, ∧, ∨, ⇒, ⇔)
KB is conjunction of its sentences
Validity and Satisfiability
Valid: True in all models (e.g., A ∨ ¬A)
Satisfiable: True in some model (e.g., A ∨ B)
Unsatisfiable: True in no models (e.g., A ∧ ¬A)
Validity ⇔ Inference (KB ⊨ α iff KB ⇒ α is valid)
Refutation/Proof by Contradiction (KB ⊨ α iff KB ∧ ¬α is unsatisfiable)
Inference Algorithms (PL)
Inference by Enumeration (Model Checking)
Resolution
Refutation Completeness (confirm/refute sentence)
Requires Conversion to CNF
Chaining (for Horn Clauses)
Horn Clause: at most one +ve literal
Definite Clause: exactly one +ve literal
Fact: no -ve literal
Forward Chaining (FC): Data-driven, irrelevant work possible, linear time
Backward Chaining (BC): Goal-driven, less complex, linear time
Pros and Cons of PL
Advantages: Declarative, Partial/Disjunctive info, Compositional, Context-independent
Disadvantages: Very limited expressive power (needs many rules)
First-Order Logic (FOL)
Ontological/Epistemological Commitment
PL: Facts hold
FOL: Objects with relations hold
FOL Basic Elements (Syntax)
Constant Symbols (KingJohn, 2)
Predicate Symbols (Brother, >)
Function Symbols (Sqrt, LeftLegOf)
Variables (x, y)
Connectives (¬, ⇒, ∧, ∨, ⇔)
Equality (=)
Quantifiers (∀, ∃)
Quantifiers
Universal (∀ 'for all')
Statement about all objects with certain properties
Natural connective: ⇒
Existential (∃ 'there exists')
Statement about some unnamed object
Natural connective: ∧
Duality (∀x P ⇔ ¬∃x ¬P)
FOL Inference Techniques
Reduction to PL
Universal Instantiation (UI): Add new sentences (logically equivalent)
Existential Instantiation (EI): Replace existential sentence (inferentially equivalent)
Inefficient (lots of irrelevant sentences)
Direct FOL Inference
Unification
Finds substitution θ (unifier) where Subst(θ, p) = Subst(θ, q)
Most General Unifier (MGU)
Generalized Modus Ponens (GMP): Sound and complete for definite clauses
Forward Chaining (FOL): Uses GMP, data-driven, complete for definite clauses
Backward Chaining (FOL): Goal-driven, can suffer incompleteness
Resolution-based Inference: Refutation-complete, requires CNF
Knowledge Engineering in FOL (7 Steps)
1. Identify the task
2. Assemble relevant knowledge
3. Decide on vocabulary (predicates, functions, constants)
4. Encode general domain knowledge
5. Encode specific problem instance
6. Pose queries
7. Debug the knowledge base